

O3 CPU - Issue/Execute/Writeback (IEW) Stage 详细分析

概述

IEW 阶段是 O3 CPU 流水线后端的核心，包含三个子阶段：

1. **Dispatch/Issue (发射)**: 将重命名后的指令分发到指令队列 (IQ) 和加载存储队列 (LSQ)，并在操作数就绪时发射指令到功能单元
2. **Execute (执行)**: 指令在功能单元中执行，产生计算结果或发起内存访问
3. **Writeback (写回)**: 将执行结果写回物理寄存器文件，唤醒依赖指令，并发送到 Commit 阶段

IEW 阶段实现了真正的乱序执行：指令按照数据就绪的顺序发射和执行，而非程序顺序。

一、IEW 阶段架构

1.1 类定义 (iew.hh)

```
class IEW
{
public:
    enum Status {
        Active,        // 阶段活跃
        Inactive       // 阶段不活跃
    };

    enum StageStatus {
        Running,        // 正常运行
        Blocked,        // 被阻塞
        Idle,           // 空闲
        StartSquash,     // 开始 squash
        Squashing,      // 正在 squash
        Unblocking,     // 正在解阻塞
        ThreadStatusMax
    };

private:
    Status _status;
    StageStatus dispatchStatus[MaxThreads];
    StageStatus exeStatus;    // 执行状态
    StageStatus wbStatus;    // 写回状态

    // 指令队列
    std::queue<DynInstPtr> insts[MaxThreads];
    std::queue<DynInstPtr> skidBuffer[MaxThreads];

    // 核心资源
    InstructionQueue instQueue;    // 指令队列（非内存指令）
}
```

```
LSQ ldstQueue;           // 加载存储队列
Scoreboard *scoreboard;   // 记分板
std::vector<FUPool *> fuPools; // 功能单元池

// 状态标志
bool wroteToTimeBuffer;
bool updateLSQNextCycle;
bool updatedQueues;
bool fetchRedirect[MaxThreads];
bool serializeInst[MaxThreads];

// 配置参数
const Cycles commitToIEWDelay;
const Cycles renameToIEWDelay;
const Cycles issueToExecuteDelay;
const unsigned dispatchWidth; // 分发宽度
const unsigned issueWidth;    // 发射宽度
const unsigned wbWidth;       // 写回宽度
const unsigned skidBufferMax;

};
```

1.2 子阶段状态说明

状态	说明
dispatchStatus[]	每个线程的分发状态
exeStatus	全局执行状态
wbStatus	全局写回状态

二、tick() 主函数

2.1 执行流程

```
void IEW::tick()
{
    bool status_change = false;

    // 1. 检查信号并更新各线程状态
    for (ThreadID tid = 0; tid < numThreads; tid++) {
        status_change = status_change || checkSignalsAndUpdate(tid);
    }

    // 2. Dispatch: 将指令分发到 IQ/LSQ
    for (ThreadID tid = 0; tid < numThreads; tid++) {
        dispatch(tid);
    }

    // 3. Execute: 从 IQ 获取就绪指令并执行
    executeInsts();
}
```

```

// 4. Writeback: 将执行结果写回寄存器
writebackInsts();

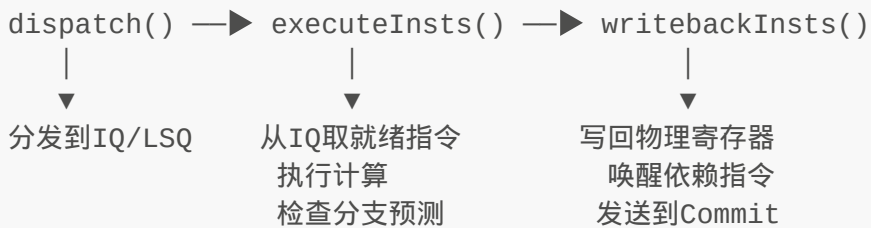
// 5. 更新时间缓冲
issueToExecuteQueue.advance();
iewQueue->advance();

// 6. 更新总体状态
if (status_change) {
    updateStatus();
}
}

```

2.2 三个子阶段执行顺序

在同一周期内，三个阶段按以下顺序执行：



三、Dispatch 子阶段

3.1 dispatch() 函数

```

void IEW::dispatch(ThreadID tid)
{
    if (dispatchStatus[tid] == Running) {
        dispatchInsts(tid);
    } else if (dispatchStatus[tid] == Unblocking) {
        if (skidBuffer[tid].empty()) {
            dispatchStatus[tid] = Running;
            toRename->iewInfo[tid].used = true;
            wroteToTimeBuffer = true;
        }
    } else if (dispatchStatus[tid] == Squashing) {
        if (squashInst[tid] == nullptr) {
            dispatchStatus[tid] = Running;
        }
    }
}

```

3.2 dispatchInsts() - 分发指令到 IQ/LSQ

```

void IEW::dispatchInsts(ThreadID tid)
{
    unsigned num_insts = std::min(
        static_cast<unsigned>(insts[tid].size()),
        dispatchWidth);

    for (int i = 0; i < num_insts; i++) {
        DynInstPtr inst = insts[tid].front();
        insts[tid].pop();

        if (inst->isSquashed()) {
            stats.dispSquashedInsts++;
            continue;
        }

        // --- 分发到不同的队列 ---
        if (inst->isLoad()) {
            // 加载指令 → LSQ
            ldstQueue.insertLoad(inst);
            stats.dispLoadInsts++;
        } else if (inst->isStore()) {
            // 存储指令 → LSQ
            ldstQueue.insertStore(inst);
            stats.dispStoreInsts++;
        } else if (inst->isMemBarrier()) {
            // 内存屏障 → IQ (作为屏障指令)
            instQueue.insertBarrier(inst);
        } else if (!inst->isNoop()) {
            // 普通指令 → IQ
            instQueue.insert(inst);
        }

        if (!inst->isNonSpeculative()) {
            stats.dispNonSpecInsts++;
        }
    }

    stats.dispatchedInsts += num_insts;

    if (num_insts > 0) {
        updatedQueues = true;
    }
}

```

3.3 分发路径

Rename 输出的指令

```

|
├─ isLoad()  ─► ldstQueue.insertLoad()
├─ isStore() ─► ldstQueue.insertStore()

```

```

└─ isMemBarrier() —► instQueue.insertBarrier()
└─ isNoop() —► 丢弃
└─ 其他 —► instQueue.insert()

```

四、Execute 子阶段

4.1 executeInsts() 函数

```

void IEW::executeInsts()
{
    // --- 从 IQ 获取就绪指令并执行 ---
    DynInstPtr inst;
    while ((inst = instQueue.getInstToExecute())) {
        Fault fault = inst->execute();

        if (fault != NoFault) {
            DPRINTF(IEW, "Execute fault: %s\n", fault.name());

            if (inst->isAtomic()) {
                // 原子操作错误需要 squash
                squashDueToFault(inst, inst->threadNumber);
                break;
            } else {
                DPRINTF(IEW, "Fault in non-atomic inst: %s\n",
                    inst->threadNumber, fault.name());
            }
        }
    }

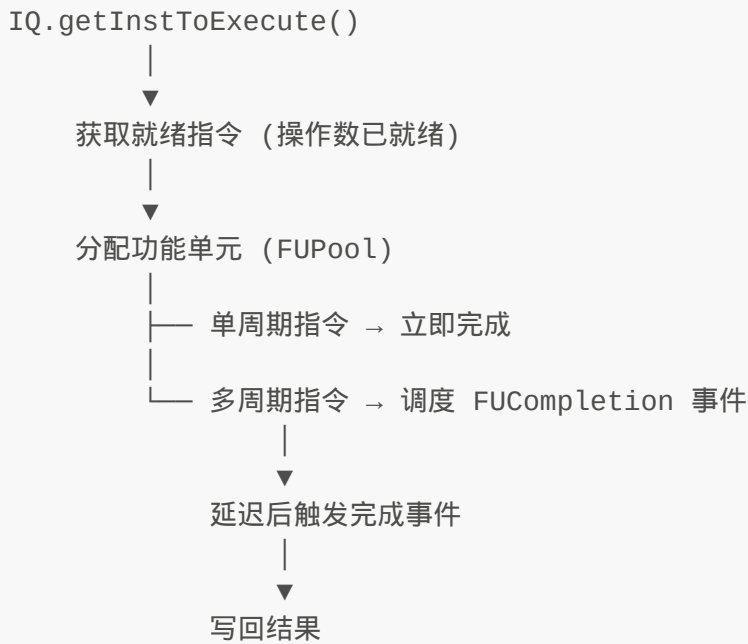
    // --- 分支预测检查 ---
    if (inst->isControl()) {
        if (inst->predTaken() != inst->traceData.taken ||
            inst->pcState().nextPC() != inst->traceData.branchTarget) {
            // 分支预测错误
            checkMisprediction(inst);
        }
    }
}

// --- 处理 LSQ 执行 ---
ldstQueue.executeInsts();

// --- 检查内存顺序违规 ---
if (ldstQueue.violation()) {
    DynInstPtr violator = ldstQueue.getMemDepViolator(
        inst->threadNumber);
    squashDueToMemOrder(violator, violator->threadNumber);
}
}

```

4.2 指令执行流程



4.3 IQ 调度就绪指令

```

void InstructionQueue::scheduleReadyInsts()
{
    unsigned to_execute = 0;

    while (to_execute < totalWidth) {
        DynInstPtr inst = nullptr;

        // 按年龄顺序从就绪队列获取指令
        if (!listOrder.empty()) {
            ListOrderEntry entry = listOrder.front();
            inst = getInstToExecute(entry.queueType);

            if (inst == nullptr) {
                moveToYoungerInst(listOrder.begin());
                continue;
            }
            listOrder.pop_front();
        } else {
            break;
        }

        // 查找可用的功能单元
        FUPool *fu_pool = findFU(inst);

        if (fu_pool == nullptr) {
            stats.fuBusy[inst->opClass()]++;
            break;
        }
    }
}
  
```

```

    int fu_idx = fu_pool->getUnit(inst->opClass());

    if (fu_idx == -1) {
        stats.fuBusy[inst->opClass()]++;
        break;
    }

    // 标记指令已发射
    inst->setIssued();

    // 根据操作延迟处理
    Cycles op_lat = fu_pool->getOpLatency(inst->opClass());

    if (op_lat == 1) {
        // 单周期指令，立即完成
        processFUCompletion(inst, fu_pool, fu_idx);
    } else {
        // 多周期指令，调度延迟完成事件
        FUCompletion *event = new FUCompletion(
            inst, fu_pool, fu_idx, this);
        schedule(event, clockEdge(op_lat - 1));
    }

    to_execute++;
}

stats.numIssuedDist.sample(to_execute);
}

```

4.4 FUCompletion - 功能单元完成事件

```

class FUCompletion : public Event
{
    DynInstPtr inst;
    FUPool *fuPool;
    int fuIdx;
    InstructionQueue *iqPtr;
    bool freeFU;

    virtual void process()
    {
        // 执行指令
        inst->execute();

        // 写回结果
        iqPtr->writeback(inst);

        // 释放功能单元
        fuPool->freeUnitNextCycle(fuIdx);
    }
}

```

```
    }  
};
```

4.5 分支预测检查

```
void IEW::checkMisprediction(const DynInstPtr &inst)  
{  
    ThreadID tid = inst->threadNumber;  
  
    DPRINTF(IEW, "[tid:%i] Branch misprediction at PC %#x.\n",  
            tid, inst->pcState().instAddr());  
  
    // 发起 squash  
    squashDueToBranch(inst, tid);  
  
    // 更新分支预测器  
    bpu->update(inst->seqNum, inst->predTaken(), inst->pcState(), tid);  
  
    stats.branchMispredicts++;  
}
```

五、Writeback 子阶段

5.1 writebackInsts() 函数

```
void IEW::writebackInsts()  
{  
    unsigned wb_insts = 0;  
  
    // 写回指令, 受 wbWidth 限制  
    while (wb_insts < wbWidth) {  
        DynInstPtr inst;  
  
        // 从执行完成的指令中获取  
        inst = getReadyInst();  
  
        if (inst == nullptr) break;  
  
        // --- 写回寄存器 ---  
        for (int i = 0; i < inst->numDestRegs(); i++) {  
            PhysRegIdPtr dest_reg = inst->destReg(i);  
            scoreboard->setReg(dest_reg); // 标记寄存器就绪  
        }  
  
        // --- 唤醒依赖指令 ---  
        wakeDependents(inst);  
  
        // --- 标记指令已完成 ---  
        wb_insts++;  
    }  
}
```



```

        inst->setCompleted();

        // --- 发送到 Commit ---
        instToCommit(inst);

        wb_insts++;
    }

    stats.writebackCount.sample(wb_insts);
}

```

5.2 wakeDependents() - 唤醒依赖指令

```

void IEW::wakeDependents(const DynInstPtr &inst)
{
    // 对于每个目的寄存器
    for (int i = 0; i < inst->numDestRegs(); i++) {
        PhysRegIdPtr dest_reg = inst->destReg(i);

        // 在依赖图中查找等待该寄存器的指令
        // 并将它们标记为就绪
        instQueue.wakeWaitingInsts(dest_reg);
    }
}

```

唤醒机制是乱序执行的关键：

1. 当一条指令写回结果时
2. 所有依赖该结果的指令被标记为"就绪"
3. 就绪的指令可以被 IQ 调度执行

六、InstructionQueue (指令队列)

6.1 IQ 类结构

```

class InstructionQueue
{
private:
    // 指令列表 (按线程)
    std::list<DynInstPtr> instList[MaxThreads];

    // 就绪执行队列
    std::list<DynInstPtr> instsToExecute;

    // 延迟的内存指令
    std::list<DynInstPtr> deferredMemInsts;
    std::list<DynInstPtr> blockedMemInsts;
    std::list<DynInstPtr> retryMemInsts;
}

```

```

// 按操作类型分类的就绪队列（优先级队列）
struct PqCompare {
    bool operator()(const DynInstPtr &lhs,
                    const DynInstPtr &rhs) const;
};
typedef std::priority_queue<
    DynInstPtr, std::vector<DynInstPtr>, PqCompare> ReadyInstQueue;
ReadyInstQueue readyInsts[Num_OpClasses];

// 非推测指令
std::map<InstSeqNum, DynInstPtr> nonSpecInsts;

// 依赖图
DependencyGraph<DynInstPtr> dependGraph;

// 内存依赖单元
MemDepUnit memDepUnit[MaxThreads];

// 年龄顺序列表
struct ListOrderEntry {
    OpClass queueType;
    InstSeqNum oldestInst;
};
std::list<ListOrderEntry> listOrder;
ListOrderIt readyIt[Num_OpClasses];
bool queueOnList[Num_OpClasses];
};

```

6.2 insert() - 插入指令

```

void InstructionQueue::insert(const DynInstPtr &new_inst)
{
    ThreadID tid = new_inst->threadNumber;
    OpClass op_class = new_inst->opClass();

    // 添加到指令列表
    instList[tid].push_back(new_inst);
    new_inst->setInIQ();

    // 添加到依赖图
    addToDependents(new_inst);
    addToProducers(new_inst);

    // 检查是否就绪（所有源操作数已就绪）
    addIfReady(new_inst);
}

```

6.3 依赖图

IQ 使用依赖图跟踪指令间的数据依赖：

依赖图结构：

每个物理寄存器 → 等待该寄存器的指令列表

```
p10 → [I3, I5, I8]    // I3, I5, I8 等待 p10 的值
p11 → [I6]            // I6 等待 p11 的值
p12 → []              // 无等待
```

当 p10 被写入时：

- I3, I5, I8 的源操作数计数减 1
- 如果某指令所有源操作数就绪 → 移入就绪队列

6.4 getInstToExecute() - 获取可执行指令

```
DynInstPtr InstructionQueue::getInstToExecute()
{
    // 按年龄顺序（最老的优先）从就绪队列获取
    for (auto it = listOrder.begin(); it != listOrder.end(); ++it) {
        DynInstPtr inst = getInstToExecute(it->queueType);
        if (inst) {
            return inst;
        }
        moveToYoungerInst(it);
    }
    return nullptr;
}
```

七、LSQ - 加载存储队列

7.1 LSQ 类结构

```
class LSQ
{
private:
    std::vector<LSQUnit> thread;    // 每线程 LSQ 单元

    DcachePort dcachePort;         // 数据缓存端口

    unsigned LQEntries;             // 加载队列大小
    unsigned SQEntries;             // 存储队列大小

    const bool recvRespThrottling;
    const unsigned recvRespMaxCachelines;
    const unsigned recvRespBufferSize;
};
```

7.2 LSQUnit - 每线程单元

```
class LSQUnit
{
private:
    std::deque<LQEntry> loadQueue;    // 加载队列
    std::deque<SQEntry> storeQueue;   // 存储队列

    bool cacheBlocked;
    int storeOffset;
    int loadOffset;

    bool storeWaitingOnWB;
    bool loadWaitingOnWB;
    bool loadBlockedOnStore;

    // 内存依赖
    DynInstPtr memDepViolator;    // 内存顺序违规者
};
```

7.3 LSQRequest - 内存请求

```
class LSQRequest : public BaseMMU::Translation
{
    enum class State {
        NotIssued,
        Translation,
        Request,
        Fault,
        PartialFault,
    };

    enum Flag : FlagsStorage {
        IsLoad = 0x00000001,
        WriteBackToRegister = 0x00000002,
        Delayed = 0x00000004,
        IsSplit = 0x00000008,
        TranslationStarted = 0x00000010,
        TranslationFinished = 0x00000020,
        Sent = 0x00000040,
        Retry = 0x00000080,
        Complete = 0x00000100,
        TranslationSquashed = 0x000000200,
        Discarded = 0x000000400,
        LSQEntryFreed = 0x000000800,
        WritebackScheduled = 0x000001000,
        WritebackDone = 0x000002000,
        IsAtomic = 0x000004000
    };
};
```

```

    FlagsType flags;
    State _state;
    LSQUnit& _port;
    const DynInstPtr _inst;
    std::vector<PacketPtr> _packets;
    std::vector<RequestPtr> _reqs;
    std::vector<Fault> _fault;
};

```

7.4 内存请求流程

Load 指令:

insertLoad() → TLB 翻译 → 发送 D-cache 读请求 → 接收响应 → 写回数据

Store 指令:

insertStore() → TLB 翻译 → 等待 Commit →
Commit 后发送到 D-cache 写请求

原子操作:

insertLoad() → TLB 翻译 → 发送 D-cache 原子操作请求 → 接收响应 → 写回

7.5 内存顺序违规检测

```

bool LSQUnit::violation() const
{
    // 检查加载是否从较新的存储获取了错误的的数据
    // 即加载在存储之前执行，但应该看到存储后的值
    return memDepViolator != nullptr;
}

```

当检测到内存顺序违规时:

1. 触发 squash，从违规的加载指令开始
2. 重新取指和执行
3. 确保加载在正确的存储之后执行

八、FUPool - 功能单元池

8.1 类结构

```

class FUPool
{
    class FUIdxQueue {
        std::deque<int> freeList; // 空闲功能单元索引
    }
};

```

```

        int capacity;
        int getUnit();
        void freeUnitNextCycle(int idx);
    };

    struct FUDesc {
        std::string name;
        int count;          // 该类型 FU 数量
        unsigned opLat;     // 操作延迟
        OpClass opClass;    // 支持的操作类型
    };

    std::vector<int> fuIndices;
    std::vector<bool> busy;
    std::vector<unsigned> opLatencies;
    std::vector<OpClass> opClasses;
    std::vector<FUIdxQueue> fuQueues;

    int getUnit(OpClass op_class);
    void freeUnitNextCycle(int idx);
    bool isCapable(OpClass op_class);
};

```

8.2 功能单元分配

```

请求操作: ALU_Add
|
▼
查找支持 ALU_Add 的 FU → FUIdxQueue
|
├── 有空闲 FU → 分配, 标记 busy
|
└── 全部忙 → 返回 -1, 指令等待

```

九、阶段间通信

9.1 输入 (来自 Rename)

```

struct RenameStruct {
    struct {
        DynInstPtr insts[MaxRenameWidth];
        int size;
        ThreadID tid;
    } renameInfo[MaxThreads];
};

```

通过 `renameQueue` 时间缓冲接收已重命名指令。

9.2 输出（到 Commit）

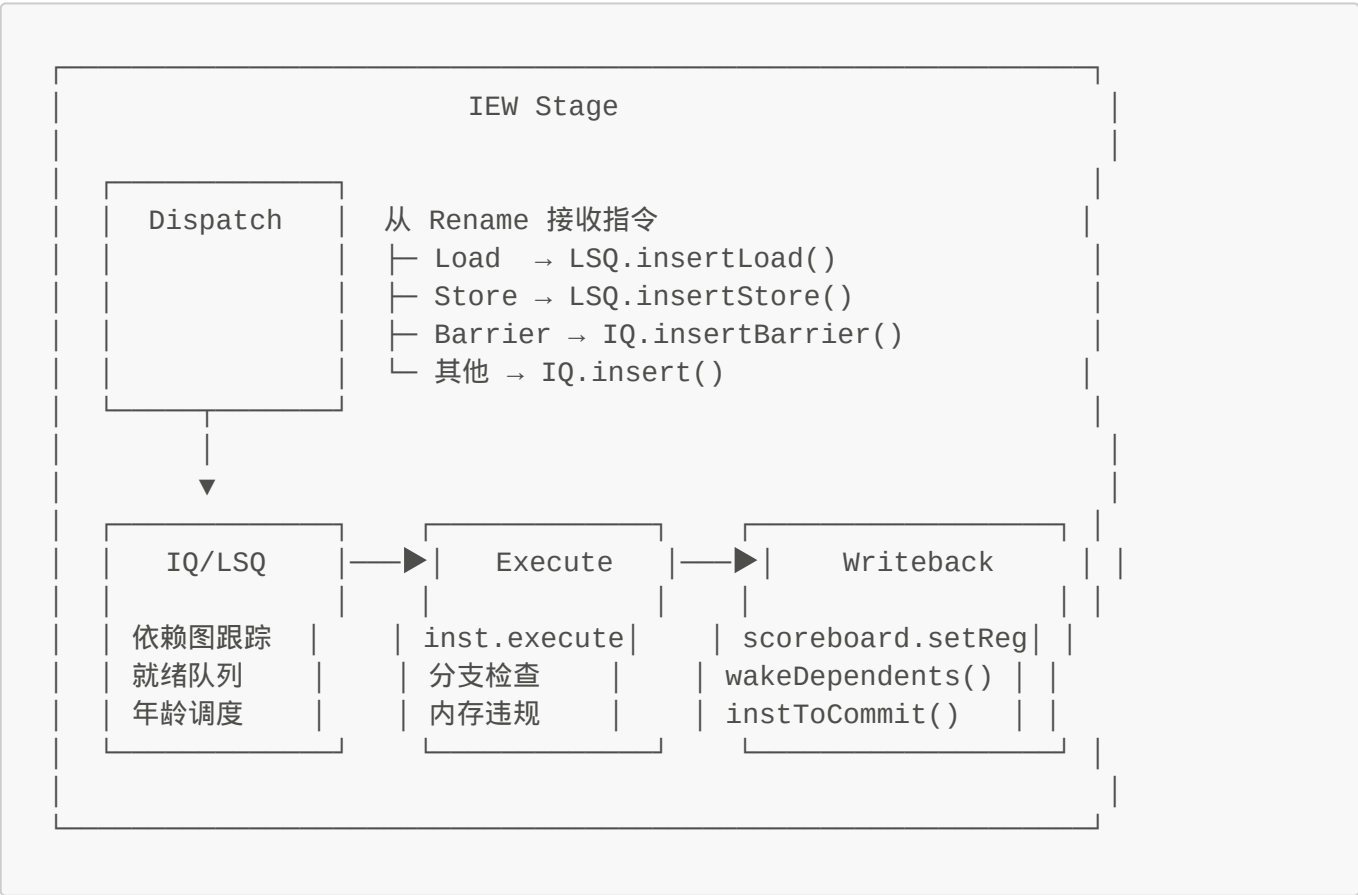
```
struct IEWStruct {
    struct {
        DynInstPtr insts[MaxIEWWidth];
        int size;
        ThreadID tid;
    } iewInfo[MaxThreads];
};
```

通过 `iewQueue` 时间缓冲发送已写回指令。

9.3 反向信号

发送方	信号	接收方	动作
IEW	squash	Decode/Rename	squash 对应线程
IEW	squash	Fetch/BAC	squash 并更新 PC
Commit	squash	IEW/Decode/Rename	squash
Commit	trap	Fetch/IEW	处理 trap

十、整体 IEW 数据流



十一、关键参数

参数	说明
dispatchWidth	每周期最大分发指令数
issueWidth	每周期最大发射指令数
wbWidth	每周期最大写回指令数
numIQEntries	IQ 总条目数
LQEntries	加载队列条目数
SQEntries	存储队列条目数
skidBufferMax	滑移缓冲区最大容量
renameToIEWDelay	Rename 到 IEW 延迟
commitToIEWDelay	Commit 反传到 IEW 延迟

十二、关键设计要点

12.1 乱序执行机制

IEW 阶段通过以下机制实现乱序执行：

- 1. **依赖跟踪**: IQ 的依赖图跟踪每条指令的源操作数是否就绪
- 2. **就绪队列**: 按操作类型分类的就绪指令优先级队列
- 3. **年龄调度**: 在同类型指令中，最老的指令优先发射（避免饥饿）
- 4. **功能单元分配**: 根据操作类型动态分配可用的功能单元
- 5. **延迟完成**: 多周期操作通过事件调度延迟完成

12.2 内存操作的推测执行

- **Load 指令**: 推测执行，不等待之前的 Store 完成
- **Store 指令**: 地址和数据在 IEW 阶段计算，但实际写入内存等到 Commit 阶段
- **内存顺序违规**: 通过 Load-Store 比较检测，违规时触发 squash

12.3 分支预测验证

在执行阶段验证分支预测：

- 1. 控制指令执行后，比较预测结果与实际结果
- 2. 如果不匹配，触发 squash，从正确的分支重新取指

生成信息

- **分析范围**: IEW Stage (iew.hh/cc, inst_queue.hh/cc, lsq.hh/cc, fu_pool.hh/cc)
- **gem5 版本**: v25.1.0.0